🎟 Ticketing System

This project generates, prints, and scans tickets with QR codes. It uses Python, Flask, ReportLab, and PyPDF2. The system is now deployable both locally and via a public server using Nginx and HTTPS.

Objective

- Automate creation of event tickets with unique QR codes.
- Provide a secure local validation system accessible via mobile devices.
- Ensure smooth scanning and validation at event entrances without relying on third-party services.

Methodology

- **Ticket generation**:
  - Configurable event name and number of tickets in `config.py`.
  - Templates defined as PDFs, overlaid with unique QR codes.
  - Ticket data stored in SQLite (`tickets.db`), with backup for each event.
- **Validation server**:
  - Flask app with HTTPS (self-signed certificates).
  - Endpoints to validate tickets in real-time.
  - Automatic logging of scans and status updates.
- **Scanning page**:
  - Accessible via QR code linking to `https://<server-ip>:8000/scan`.
  - Uses ZXing JS for live camera decoding.
  - Displays clear status messages (valid, already used, invalid).
  - 3-second buffer between scans.

Usage Workflow

1. **Generate tickets**
   - Edit `config.py` (event name, ticket count, template path).
   - Run `python generate_tickets.py`.
   - Outputs: tickets with QR codes, database, and CSV with ticket URLs.
2. **Start validation server**
   - Run `python app.py` (or `flask run` with certificate and key).
   - Server available at `https://<LAN-IP>:8000/`.
3. **Scan tickets**
   - Staff connect to event Wi-Fi, open scan QR code in browser.
   - Accept certificate and allow camera access.

- Page shows validation result in large colored text.

For Developers

1. Prerequisites

Local System Requirements (for generating tickets)

- Python 3.8+ installed (Windows or Linux)
- Git and OpenSSL (for creating SSL certs if needed)
- On Windows: allow inbound TCP port 8000 in Windows Firewall
- Make sure all devices are connected to the **same network** when testing locally

VPS Server Requirements (for public deployment)

- A public VPS (e.g. Hetzner Cloud) with Ubuntu 22.04
- A domain name (e.g. `tickets.danielnetzl.com`)
- Domain DNS pointing to your VPS (A record)
- `sudo` access via SSH (preferably using SSH keys)

---

2. Clone & Install (on any system)

```
git clone <repo-url> ticketing_system
cd ticketing_system
python -m venv venv
source venv/bin/activate  # Or .venv\Scripts\activate on Windows
pip install -r requirements.txt
```

---

3. Generate Tickets

1. Edit `config.py` and set:
    - `EVENT` (name of the event)
    - `N_TICKETS`
    - `BASE_PATH` and other layout config if needed
    - `URL = "tickets.danielnetzl.com"` (no prefix)
2. Place your event template PDF into `ticket_templates/` named as `{EVENT}.pdf`.
3. Generate everything (QR codes, PDFs, database, backup):

```
python generate_tickets.py
```

🎉 This creates:

- `events/{EVENT}/tickets.db`
- `qr_codes/` + `scan_page.png`
- `ticket_urls.csv`
- `tickets/` as printable PDFs
- A full backup in `bkp/{EVENT}`

---

4. Deploy to VPS with Nginx + HTTPS

🔐 A. Set up server

On your VPS:

```
sudo apt update
sudo apt install nginx python3-pip python3-venv git ufw -y
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw enable
```

📁 B. Clone your project and set up Python

```
adduser tickets
usermod -aG sudo tickets
su - tickets
git clone <repo> ticketing_system
cd ticketing_system
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

🚀 C. Run the app via systemd

Create `/etc/systemd/system/tickets.service`:

```
[Unit]
Description=Gunicorn for Ticket App
After=network.target

[Service]
User=tickets
Group=www-data
WorkingDirectory=/home/tickets/ticketing_system
Environment="PATH=/home/tickets/ticketing_system/venv/bin"
ExecStart=/home/tickets/ticketing_system/venv/bin/gunicorn -w 4 -b 127.0.0.1:8000
app:app
```

```
[Install]
WantedBy=multi-user.target
```

Then:

```
sudo systemctl daemon-reexec
sudo systemctl enable tickets
sudo systemctl start tickets
```

Check with:

```
sudo systemctl status tickets
```

D. Configure Nginx

Edit `/etc/nginx/sites-available/tickets`:

```
server {
    listen 80;
    server_name tickets.danielnetzl.com;

    location / {
        proxy_pass http://127.0.0.1:8000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Then:

```
sudo ln -s /etc/nginx/sites-available/tickets /etc/nginx/sites-enabled/
sudo rm /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
```

E. Enable HTTPS with Let's Encrypt

```
sudo apt install certbot python3-certbot-nginx -y
sudo certbot --nginx
```

Choose to redirect HTTP to HTTPS when prompted.

Now your app is securely available at:

```
https://tickets.danielnetzl.com/scan
```

---

5. Scanner Webpage

- URL: `https://tickets.danielnetzl.com/scan`
- QR code for this page is saved as `scan_page.png`
- Uses live camera access (requires HTTPS)
- 3-second delay between scans to avoid double scans

---

6. Instructions for Entrance Staff

This guide is for staff scanning tickets on phones/tablets.

☑ Setup

1. **Use Safari (iOS)** or **Chrome (Android)**. Ecosia, Firefox, DuckDuckGo won't work.
2. Connect to mobile data or event Wi-Fi.
3. Scan the `scan_page.png` QR code and open the link in Safari/Chrome.
4. When prompted:
    - Allow camera access
    - Accept HTTPS certificate (if needed)

> Tip: Tap **Share** → **Zum Home-Bildschirm** to pin it as a full-screen app.

🎯 During the Event

- Hold the QR code clearly in the camera frame
- A message will appear:
    - ☑ **Ticket gültig** → allow entry
    - ❌ **Bereits verwendet** → reject
    - ❌ **Ungültig** → reject
- Wait 3 seconds before the next scan

🛠 Troubleshooting

- **Camera not available**: Make sure you use Safari or Chrome + HTTPS
- **Scan not detected**: Check lighting, glare, or focus
- **Tab closes**: Re-open the scan URL, clear browser cache if needed

---

7. Developer Notes

- Run `python generate_tickets.py` whenever you want to:
  - Create a new event
  - Regenerate QR codes
- All output is placed under `events/{EVENT}` and backed up
- The app auto-loads the correct database for the current event

---

☑ You're all set!

Access:

```
https://tickets.danielnetzl.com/scan
```

Support: Ask Daniel if anything breaks.